实验5 内存管理

- 实验目的
  - 了解内存管理的概念，掌握分页、分段操作过程
  - 掌握虚拟内存技术的概念，重点理解替换策略
  - 理解选择替换页算法OPT、FIFO、LRU、CLOCK

实验内容与要求

（用户态）在Linux下编程实现虚存页面替换算法的模拟程序。主要包含：

- 根据用户输入参数，包含进程大小（页数），进程地址（页地址）顺序，页表大小
- 验证test5.c中FIFO、LRU替换算法。
- 新建工程replace，编写并运行OPT和CLOCK算法。

预备知识（1）

进程大小（页数）

进程大小是进程地址空间的可寻址长度。地址空间（address space）表示一个进程占用的内存大小。例如32位计算机中内存地址由32位无符号整数表示，可寻址范围为2^32 = 4GB的长度，即可用内存大小4GB。现假设该计算机的页大小为1K，则一个大小为5K的程序可以分为5页，即页数为5。

预备知识（2）

页地址访问顺序

页地址访问顺序是进程执行过程中访问页的顺序。假设页大小为1K，一个大小为5K的进程，其数据和指令存储在5个页中，为其分别编号为1，2，3，4，5。在进程执行过程中，需要访问编号为1~5页中的数据或者指令，故一个可能的访问页地址顺序为：

2 3 2 1 5 2 4 5 3 2 5 2

注意：访问的顺序和进程指令的执行有关，故上述只是一个可能的顺序。页地址是逻辑地址。

预备知识（3）

页表

页表给出了该进程每一页对应于物理内存帧的位置。由于内存空间被许多进程共享，如果把一个进程全部加载进内存，会导致内存资源非常紧张，限制了进程并发运行的数量。内存管理使得只有一部分进程加载到内存中，页表形成被加载的内存页与实际的物理内存地址帧之间的映射。内存的加载过程涉及内存的替换策略。

示例

一个大小为5K的进程，页大小为1K，可能出现如下情况：

页数：5

页地址顺序：1 | 2 | 1 | 5 | 3 | 4 | 3 | 2 | 1 | 5 |

驻留集大小： 3

预备知识（4）

FIFO （先入先出算法， Fist In Fist Out）

最简单的页面置换算法是先入先出(FIFO)法。先入先出法的实质是，总是选择在主存中停留时间最长(即最老)的一页置换，即先进入内存的页，先退出内存。

这种算法的核心思想是认为最早调入内存的页，其不再被使用的可能性比刚调入内存的可能性大。建立一个FIFO队列，收容所有在内存中的页。被置换页面总是在队列头上进行。当一个页面被放入内存时，就把它插在队尾上。

示例

页数：10

页地址顺序： | 4 | 8 | 0 | 1 | 8 | 3 | 9 | 4 | 1 | 0 | 4 | 1 | 2 | 9 | 7 | 9 | 3 | 3 | 3 | 2 | 0 | 9 | 9 | 1 | 6 | 2 | 1 | 9 | 5 | 6 |(从0开始)

驻留集大小： 3

OPT替换结果：

no. 0 step: | 4 |
no. 1 step: | 4 | 8 |
no. 2 step: | 4 | 8 | 0
no. 3 step: | 1 | 8 | 0
no. 4 step: | 1 | 8 | 0
no. 5 step: | 1 | 3 | 0
no. 6 step: | 1 | 3 | 9
no. 7 step: | 4 | 3 | 9
no. 8 step: | 4 | 1 | 9
no. 9 step: | 4 | 1 | 0
no. 10 step: | 4 | 1 | 0
no. 11 step: | 4 | 1 | 0
no. 12 step: | 2 | 1 | 0
no. 13 step: | 2 | 9 | 0
no. 14 step: | 2 | 9 | 7
no. 15 step: | 2 | 9 | 7
no. 16 step: | 3 | 9 | 7
no. 17 step: | 3 | 9 | 7
no. 18 step: | 3 | 9 | 7
no. 19 step: | 3 | 2 | 7
no. 20 step: | 3 | 2 | 0
no. 21 step: | 9 | 2 | 0
no. 22 step: | 9 | 2 | 0
no. 23 step: | 9 | 1 | 0
no. 24 step: | 9 | 1 | 6
no. 25 step: | 2 | 1 | 6
no. 26 step: | 2 | 1 | 6
no. 27 step: | 2 | 9 | 6
no. 28 step: | 2 | 9 | 5
no. 29 step: | 6 | 9 | 5

缺页次数：19次

预备知识（6）

LRU （最近最久未使用算法， Least Recently Used ）

LRU算法的提出，是基于这样一个事实：在前面几条指令中使用频繁的页面很可能在后面的几条指令中频繁使用。反过来说，已经很久没有使用的页面很可能在未来较长的一段时间内不会被用到。

核心思想是，当需要置换一页时，选择在最近一段时间里最久没有使用过的页面予以置换。既然要以时间为替换依据，那么肯定就要用到定时/计数器，因此这个算法通常需要一定的硬件支持，这样的话其通用性也会降低。做实验时可以用加权的方式模拟时间。

示例

页数：10

页地址顺序： | 4 | 8 | 0 | 1 | 8 | 3 | 9 | 4 | 1 | 0 | 4 | 1 | 2 | 9 | 7 | 9 | 3 | 3 | 3 | 2 | 0 | 9 | 9 | 1 | 6 | 2 | 1 | 9 | 5 | 6 |(从0开始)

页表大小： 3

OPT替换结果：

no. 0 step: | 4 |
no. 1 step: | 4 | 8 |
no. 2 step: | 4 | 8 | 0
no. 3 step: | 1 | 8 | 0
no. 4 step: | 1 | 8 | 0
no. 5 step: | 1 | 8 | 3
no. 6 step: | 9 | 8 | 3
no. 7 step: | 9 | 4 | 3
no. 8 step: | 9 | 4 | 1
no. 9 step: | 0 | 4 | 1
no. 10 step: | 0 | 4 | 1
no. 11 step: | 0 | 4 | 1
no. 12 step: | 2 | 4 | 1
no. 13 step: | 2 | 9 | 1
no. 14 step: | 2 | 9 | 7
no. 15 step: | 2 | 9 | 7
no. 16 step: | 3 | 9 | 7
no. 17 step: | 3 | 9 | 7
no. 18 step: | 3 | 9 | 7
no. 19 step: | 3 | 9 | 2
no. 20 step: | 3 | 0 | 2
no. 21 step: | 9 | 0 | 2
no. 22 step: | 9 | 0 | 2
no. 23 step: | 9 | 0 | 1
no. 24 step: | 9 | 6 | 1
no. 25 step: | 2 | 6 | 1
no. 26 step: | 2 | 6 | 1
no. 27 step: | 2 | 9 | 1
no. 28 step: | 5 | 9 | 1
no. 29 step: | 5 | 9 | 6

缺页次数：19次

预备知识（5）

OPT (最优置换算法, OPTimal)

当调入新的一页而必须预先置换某个老页时，所选择的老页应是将来不再被使用，或者是在最远的将来才被访问。

这是一种最优的算法，因为对于任一页面请求序列，其产生的缺页中断次数时最少的。

预备知识（6）

Clock （时钟算法）

Clock算法是只需为每页设置一位访问位，再将内存中的所用页面都通过链接指针链接成一个循环队列。当某页被访问时，其访问位被置1。置换算法在选择一页淘汰时，只需检查页的访问位。如果是0，就选择该页换出；若为1，则重新将他置0,暂不换出，而给该页第二次驻留内存的机会,在按照FIFO 算法检查下一个页面。当检查到队列中的最后一个页面是，若其访问位仍为1，则再返回到队首去检查第一个页面。

实验内容与要求

（用户态）在Linux下编程实现虚存页面替换算法的模拟程序。主要包含：

- 根据用户输入参数，包含进程大小（页数），进程地址（页地址）顺序，驻留集大小
- 验证test5.c中FIFO、LRU替换算法。
- 新建工程replace，编写并运行CLOCK和OPT算法。